

Predavanje 1 — Osnove UNIX-a

Programiranje za UNIX

Damir Krstinić Maja Braović

FESB — Sveučilište u Splitu

1. **Uvod**

O kolegiju

- ▶ **Cilj:** svladati osnove programiranja u UNIX okruženju, s naglaskom na korištenje **sistemskih poziva** i specifičnosti UNIX operacijskog sustava.
- ▶ Nakon kolegija znat ćete:
 - ▶ snaći se u UNIX ljusci i napisati korisnu skriptu,
 - ▶ prevesti i organizirati C program s više datoteka i biblioteka,
 - ▶ raditi s datotekama preko sistemskih poziva,
 - ▶ stvarati procese, upravljati njima i komunicirati među njima.
- ▶ Nije kolegij o *korištenju* Linuxa, nego o tome **kako sustav radi iznutra**.

Sadržaj kolegija

1. Osnove UNIX-a
2. Osnove programiranja
3. Ulazno/izlazne operacije
4. Upravljanje datotekama
5. Okruženje procesa
6. Signali
7. Komunikacija između procesa
8. *(Višenitno programiranje)*
9. *(Socketi)*

- ▶ **Skripta:** *Programiranje za UNIX*, Krstinić & Braović, FESB — devet poglavlja koja prate sadržaj predavanja.
- ▶ Repozitorij: github.com/dkrst/Programiranje_za_UNIX
 - ▶ cjeloviti tekst i PDF,
 - ▶ **svi primjeri s predavanja**, po poglavljima, spremni za prevođenje.
- ▶ Svako poglavlje završava **zadacima za samostalno rješavanje** — oni su najvažniji dio učenja: gradivo se ne može savladati samo čitanjem.
- ▶ Informacije i obavijesti: **Merlin**.

Dodatna literatura

- ▶ W. R. Stevens, S. A. Rago: *Advanced Programming in the UNIX Environment* — referentna knjiga područja.
- ▶ B. W. Kernighan, D. M. Ritchie: *The C Programming Language* ("K&R").
- ▶ P. H. Salus: *Kratka povijest UNIX-a: Od UNICS-a do FreeBSD-a i Linuxa*.
- ▶ Srce: *Osnove uporabe računala i interneta* — pregled rada u Linux okruženju: srce.unizg.hr (skripta d106).
- ▶ man stranice — najkorisniji izvor tijekom rada.

Radno okruženje

Za rad je potreban pristup UNIX ili UNIX-sličnom sustavu. Nekoliko jednako valjanih mogućnosti:

- ▶ **Linux distribucija na vlastitom računalu** — Ubuntu, Debian, Fedora, Mint; izvorno instalirana ili u virtualnom stroju (VirtualBox, VMware).
- ▶ **macOS** — terminal je odmah upotrebljiv; razvojni alati dodaju se preko Xcode Command Line Toolsa.
- ▶ **Windows** — WSL (*Windows Subsystem for Linux*) daje potpuno Linux okruženje bez virtualnog stroja.
- ▶ **Rad na udaljenom poslužitelju** — prijava preko SSH-a iz Windows *command prompt* (`ssh korisnik@posluzitelj`) ili klijentom kao što je PuTTY.

Radno okruženje — preporuka

- ▶ Za kolegij je dovoljan bilo koji od navedenih načina; svi primjeri rade jednako.
- ▶ Za prijenos datoteka na poslužitelj: scp, sftp ili grafički klijent (WinSCP, FileZilla).
- ▶ **Za one koji žele naučiti više:** instalirajte Linux na vlastito računalo — kao jedini sustav ili uz Windowse (*dual boot*).
 - ▶ Sustav se uči najbrže kad se svakodnevno koristi.
 - ▶ Vidjet ćete kako se stvari zaista postavljaju, a ne samo kako se koriste.
 - ▶ Prije instalacije: napravite sigurnosnu kopiju podataka i isprobajte distribuciju s USB-a (*live* način rada).
- ▶ Uređivač teksta po izboru: nano ili joe za početak, vi, vim ili emacs za naprednije.

Kratka povijest UNIX-a

- ▶ Razvoj počinje **1969.** u istraživačkom centru **Bell Labs** (Murray Hill, New Jersey).
- ▶ Bell Labs: 11 Nobelovih i 5 Turingovih nagrada, izum tranzistora, UNIX i programski jezik C.
- ▶ AT&T je u to vrijeme imao **legalni telefonski monopol** u SAD-u.
- ▶ Jedan od uvjeta američke vlade bio je da tvrtka **ne smije komercijalno nastupati izvan telekomunikacija** — dakle ni prodavati softver.
- ▶ Ta naizgled sporedna pravna okolnost oblikovat će cijelu daljnju povijest UNIX-a.

Od MULTICS-a do UNIX-a

- ▶ Sredinom 1960-ih Bell Labs, MIT i General Electric razvijaju **Multics** (*MULTiplexed Information and Computing Service*).
- ▶ Multics donosi napredne ideje: virtualnu memoriju, sigurnu kontrolu pristupa, dinamičko povezivanje, hijerarhijski datotečni sustav.
- ▶ Zašto je odbačen:
 - ▶ projekt je **prevelik i preambiciozan** — previše ciljeva odjednom,
 - ▶ razvoj **predugo traje**, rokovi se stalno pomiču,
 - ▶ sustav zahtijeva **previše resursa** za tadašnja računala i skup je za održavanje.
- ▶ Bell Labs 1969. izlazi iz projekta. Istraživači ostaju bez sustava za rad — i odlučuju napisati vlastiti, **manji i jednostavniji**.

Od MULTICS-a do UNIX-a

- ▶ Novi sustav dobiva ime **Unics** (*Uniplexed Information and Computing System*) — igra riječi na račun Multicsa: umjesto “multi”, “uni”.
- ▶ Ime se ubrzo piše kao **UNIX**.
- ▶ Zadržane su dobre ideje Multicsa (hijerarhijski datotečni sustav, istovremeni rad više korisnika), ali u **bitno jednostavnijoj izvedbi**.
- ▶ Prva službena verzija objavljena je **1971**.
- ▶ **1973**. UNIX je prepisan u programskom jeziku **C** — do tada su operacijski sustavi pisani gotovo isključivo u assembleru.

- ▶ **Ken Thompson** (1943.-): autor prve verzije UNIX-a; programski jezici B, C i Go; sustav Plan 9; UTF-8 enkodiranje. Anegdota: prvu je verziju napisao (i) zato da bi na jeftinijem računalu mogao pokretati svoju igru *Space Travel*.
- ▶ **Dennis Ritchie** (1941.-2011.): autor programskog jezika **C**, suautor UNIX-a. Cijelu je karijeru proveo u Bell Labsu.
- ▶ Thompson i Ritchie zajedno dobivaju **Turingovu nagradu 1983.**
- ▶ **Brian Kernighan** (1942.-): suautor knjige *The C Programming Language* ("K&R"), autor prvog programa koji ispisuje `hello, world`; kasnije profesor na Princetonu. Njemu se pripisuje i sam naziv sustava.

Sklopovlje: PDP-7

- ▶ Projekt nije imao ni službenu potporu ni proračun — Bell Labs je upravo napustio Multics i nije želio novi operacijski sustav.
- ▶ Radilo se na **odbačenom DEC PDP-7** računalu koje nitko drugi nije koristio: nekoliko desetaka kilobajta memorije, isprva bez diska, spori teleprinter umjesto zaslona.
- ▶ Sve je moralo stati u vrlo malo memorije i izvršavati se na vrlo sporom stroju.
- ▶ Posljedice vidljive do danas:
 - ▶ kratka imena naredbi (ls, cp, mv, rm),
 - ▶ mali programi koji rade jednu stvar umjesto jednog velikog sustava,
 - ▶ tekstualno sučelje i tekstualne datoteke kao univerzalni format,
 - ▶ **efikasnost kao nužnost, a ne kao stilski izbor.**

Besplatno dijeljenje i otvoreni kôd

- ▶ Zbog monopolskih ograničenja AT&T UNIX **nije smio prodavati** — pa ga je sveučilištima ustupao besplatno ili gotovo besplatno, **zajedno s izvornim kodom**.
- ▶ Tijekom sedamdesetih UNIX se širi američkim sveučilištima: studenti ga čitaju, mijenjaju i dopisuju vlastite alate.
- ▶ Nastaje kultura u kojoj je **izvorni kôd dostupan i razumljiv** — posve neuobičajeno za to doba.
- ▶ Najvažniji plod te kulture je **BSD** (*Berkeley Software Distribution*, 1977.) sa sveučilišta Berkeley: virtualna memorija, bolje upravljanje procesima, poboljšani datotečni sustav, mrežna podrška.
- ▶ Ista kultura dvadesetak godina kasnije rađa Linux i pokret otvorenog koda.

Iz jednog izvora vrlo brzo nastaje **stablo, a ne jedan sustav**:

- ▶ **Research Unix** (Bell Labs) — izvorna linija, verzije 1 do 10.
- ▶ **BSD grana** (Berkeley) — akademska i otvorena; iz nje FreeBSD, NetBSD, OpenBSD te, preko NeXTSTEP-a, **macOS**.
- ▶ **System V grana** (AT&T, nakon prestanka monopola) — komercijalna; iz nje SunOS/Solaris, IBM AIX, HP-UX, SGI IRIX, Xenix.

Osamdesete: gotovo svaki proizvođač računala nudi vlastiti UNIX — program napisan za jedan ne prevodi se na drugom.

FESB | Programiranje za UNIX



- ▶ **POSIX** (*Portable Operating System Interface*) — niz standarda koje od **1988.** objavljuje IEEE, danas usklađen sa *Single UNIX Specification* (Open Group).
- ▶ Nastao je kao odgovor na fragmentaciju: cilj je bio da se isti izvorni kôd prevede i izvršava na svakoj varijanti UNIX-a.
- ▶ Standard ne propisuje **kako** je sustav izveden iznutra, nego **što** mora nuditi prema van:
 - ▶ sistemske pozive i funkcije C biblioteke (open, fork, signal, niti...),
 - ▶ ljusku i osnovne naredbe s njihovim ponašanjem,
 - ▶ strukturu datotečnog sustava, prava pristupa, varijable okruženja.
- ▶ Sustav se može **certificirati** kao usklađen (macOS, AIX, HP-UX, Solaris); Linux i BSD nisu certificirani, ali standard u praksi slijede.
- ▶ Za nas praktično: gradivo ovog kolegija vrijedi na svim tim sustavima — učimo sučelje, ne jednu inačicu.

- ▶ **1991.:** Linus Torvalds, student u Helsinkiju, objavljuje vlastitu jezgru — isprva kao hobi projekt, nadahnut Minixom (nastavnim UNIX-om A. Tanenbauma).
- ▶ Linux je **samo jezgra**; sustav postaje upotrebljiv tek u kombinaciji s alatima projekta **GNU** (prevodilac, ljuska, osnovne naredbe) — otuda naziv GNU/Linux.
- ▶ Objavljen je pod licencom **GPL**: kôd se smije koristiti, mijenjati i dijeliti, uz uvjet da izmjene ostanu jednako otvorene.
- ▶ Nastaju **distribucije** — jezgra, alati i paketi u jednoj cjelini: Debian, Ubuntu, Fedora, Red Hat, Arch — te tvrtke koje nude komercijalnu podršku.
- ▶ Za razliku od komercijalnih UNIX-a, Linux nije vezan ni uz jednog proizvođača ni uz jednu vrstu sklopovlja — što objašnjava njegovu današnju rasprostranjenost.

- ▶ **UNIX-slični** (*UNIX-like*, **nix*) sustavi ponašaju se kao UNIX, ali nisu nužno službeno certificirani.
- ▶ Danas najvažniji: **Linux** (sve distribucije), **macOS** (certificiran, BSD temelji), **FreeBSD**, **OpenBSD**, **NetBSD**.
- ▶ Na njima počivaju i:
 - ▶ **Android** (2008.) — Linux jezgra,
 - ▶ **iOS** (2007.) — zajednički temelji s macOS-om.
- ▶ Sa stajališta programera razlike su male: svi se temelje na POSIX standardu.

Ključne osobine UNIX-a

- ▶ **Višekorisnički i višezadaćni** — sustav sam pravedno dijeli procesor i ostale resurse.
- ▶ **Modularna arhitektura** — svaki program radi jednu stvar, ali je radi dobro.
- ▶ **Ljuska** — interaktivno sučelje i ujedno potpun programski jezik.
- ▶ **Kontrola pristupa** — vlasništvo i prava nad datotekama, procesima i sklopovljem.
- ▶ **Prenosivost** — isti izvorni kôd prevodi se na različitim platformama.

UNIX poslužitelji

- ▶ Velika većina web poslužitelja, baza podataka i sustava u oblaku radi na Linuxu.
- ▶ Razlozi: stabilnost pri dugom neprekidnom radu, upravljanje na daljinu preko ljuske, automatizacija skriptama, cijena.
- ▶ Praktično: kad razvijate web aplikaciju, ona se gotovo sigurno **izvršava na UNIX-u**, neovisno o tome na čemu ste ju napisali.

Osobna računala

- ▶ **macOS** je certificirani UNIX (temeljen na BSD-u) — ispod grafičkog sučelja je standardni UNIX terminal.
- ▶ Linux distribucije na stolnim računalima: Ubuntu, Fedora, Debian, Arch...
- ▶ **WSL** (*Windows Subsystem for Linux*) donosi potpuno Linux okruženje unutar Windowsa.
- ▶ Sve što ćemo raditi na ovom kolegiju radi jednako na sva tri okruženja.

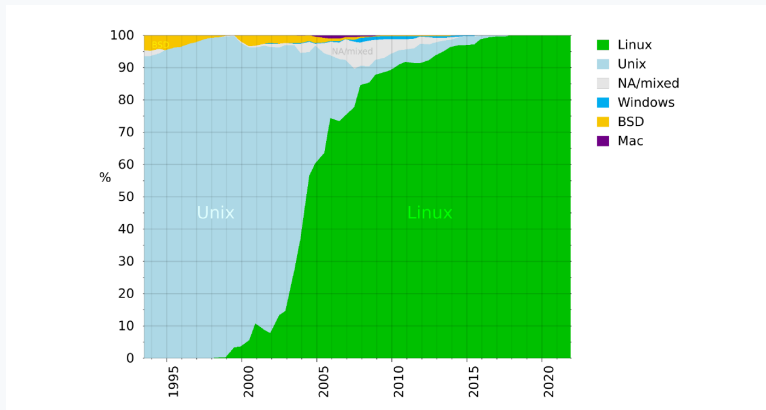
Mobilni i ugradbeni uređaji

- ▶ **Android** (Linux jezgra) i **iOS** (BSD temelji) — gotovo svi pametni telefoni na svijetu.
- ▶ Ugradbeni sustavi: usmjerivači, televizori, automobili, industrijska oprema, IoT uređaji.
- ▶ Zašto UNIX: prilagodljivost (jezgra se konfigurira prema hardveru), mali otisak, otvoreni kôd bez naknada po uređaju.

Superračunala (HPC)

- ▶ Lista **TOP500** — 500 najsnažnijih računala na svijetu.
- ▶ Od 2017. **100 % njih** koristi Linux.
- ▶ Razlozi: potpuna kontrola nad jezgrom i raspoređivanjem, podrška za specijalizirane mreže i akceleratori, prilagodljivost bez ograničavajućih licenci.

Zastupljenost na listi TOP500



Slika 1.1: Zastupljenost operacijskih sustava na TOP500 listi superračunala

Zaključak

Znanje UNIX-a nije znanje o jednom sustavu, nego o **sučelju koje se koristi posvuda** — od telefona u džepu do superračunala.

Programi koje ćemo pisati na ovom kolegiju izvršavaju se, uz najviše ponovno prevođenje, na svakom od tih uređaja.

2.

Arhitektura UNIX-a

Slojevita arhitektura



Slika 1.2: Slojevita arhitektura UNIX operacijskog sustava

Odozdo prema gore:

1. **Sklopovlje** — procesor, memorija, diskovi, uređaji.
2. **Jezgra** (*kernel*) — jedini dio s izravnim pristupom sklopovlju.
3. **Sistemske pozivi** — jedino sučelje prema jezgri.
4. **Biblioteke** — funkcije više razine iznad sistemskih poziva.
5. **Ljuska i aplikacije** — korisnički programi.

Datotečni sustav nije jedan od slojeva apstrakcije, ali je integralni dio svakog operacijskog sustava.

- ▶ Upravlja sklopovljem i dijeli resurse među korisnicima i procesima.
- ▶ Jedina ima **privilegirani pristup** sklopovlju; sve ostalo, uključujući ljusku, s hardverom komunicira posredno.
- ▶ Osigurava izolaciju: proces ne može čitati ni pisati po memoriji drugog procesa.
- ▶ Generalizira pristup: programer ne mora znati koji je točno model diska ili mrežne kartice u računalu.

- ▶ Sklopovlja ima previše i prerazličitog da bi sve bilo ugrađeno u jezgru.
- ▶ **Modul** je zaseban dio jezgre koji se može **dinamički učitati** bez ponovnog pokretanja sustava.
- ▶ Tipično su to upravljački programi za pojedine uređaje; izrađuju ih proizvođači ili zajednica.
- ▶ U Windows terminologiji ekvivalent je **driver**.

Vrste jezgri

- ▶ **Monolitne** — većina funkcija OS-a u jednom velikom programu; jednostavnije i brže.
- ▶ **Mikro jezgre** — minimalna jezgra, usluge u zasebnim procesima; problematičan dio može se zaustaviti bez rušenja sustava.
- ▶ **Hibridne** — današnji Linux i Windows: dobre strane obiju pristupa.
- ▶ **Kernel panic** — stanje u kojem jezgra ne zna kako nastaviti pa zaustavlja sustav (Windows: *blue screen of death*).

Sistemi pozivi

- ▶ Strogo definiran skup funkcija kojima korisnički program **traži uslugu od jezgre**.
- ▶ To je **jedino** sučelje između korisničkih programa i jezgre.
- ▶ Dvije razine izvršavanja:
 - ▶ **korisnički mod** — bez izravnog pristupa memoriji i sklopovlju,
 - ▶ **jezgreni mod** — puni pristup, u njemu se izvršava kôd jezgre.
- ▶ Sistemski poziv je prijelaz iz korisničkog u jezgreni mod i natrag.

Česti sistemski pozivi

Datoteke

- ▶ `open()` — otvaranje
- ▶ `read()` — čitanje
- ▶ `write()` — pisanje
- ▶ `close()` — zatvaranje

Procesi

- ▶ `fork()` — novi proces
- ▶ `exec()` — pokretanje programa
- ▶ `wait()` — čekanje djeteta
- ▶ `exit()` — završetak
- ▶ `kill()` — slanje signala

Svaki od njih obradit ćemo detaljno kroz kolegij.

- ▶ Zbirke funkcija **više razine** iznad sistemskih poziva — dodatna razina apstrakcije.
- ▶ Svaka se funkcija biblioteke u konačnici svodi na jedan ili više sistemskih poziva, ali nudi jednostavnije sučelje.
- ▶ Primjeri: libc (printf, fopen, malloc), libm (matematičke funkcije), libpthread (niti).
- ▶ **Statičke** biblioteke ugrađuju se u program pri prevođenju, **dinamičke** se povezuju pri izvršavanju.
- ▶ Uobičajena lokacija: /lib, /usr/lib.

- ▶ Interpreter naredbenog retka: čita naredbu sa standardnog ulaza, izvršava ju i ispisuje rezultat.
- ▶ **Ljuska nije dio jezgre** — to je običan korisnički program koji koristi sistemske pozive kao i svaki drugi.
- ▶ Istovremeno je i potpun programski jezik: varijable, grananje, petlje, funkcije.
- ▶ Format naredbe:

naredba [opcije] [argumenti]

man <naredba> — pomoć za bilo koju UNIX naredbu.

Standardni ulaz i izlaz

Svaki program u UNIX-u pri pokretanju dobiva tri otvorena kanala:

- ▶ **standardni ulaz** (stdin) — odakle program čita, prema zadanom s tipkovnice,
- ▶ **standardni izlaz** (stdout) — kamo program piše rezultat, prema zadanom na zaslon,
- ▶ **standardni izlaz za greške** (stderr) — kamo idu poruke o greškama, odvojeno od rezultata.

Tako komuniciramo s ljuskom, ali i sa **svakim drugim programom** — program ne mora znati je li na drugom kraju tipkovnica, datoteka ili drugi program.

Upravo zato se ta tri kanala mogu preusmjeriti ili spojiti u lance obrade — detaljno na sljedećem predavanju.

- ▶ Najviši sloj — svi korisnički programi: editori, preglednici, baze podataka, inženjerski alati.
- ▶ Sve operacije ostvaruju pozivanjem funkcija iz biblioteka ili izravnim sistemskim pozivima.
- ▶ Za sustav nema razlike između “sistemskog” i “korisničkog” programa — ls je aplikacija jednako kao i web preglednik.

3.

Datoteční sustav

Arhitektura datotečnog sustava

UNIX datotečni sustav organiziran je u obliku **stabla** s jednim korijenom /:

```
/
├── bin      osnovne naredbe (ls, cp, mv, ...)
├── dev      datoteke koje predstavljaju uređaje
├── etc      konfiguracijske datoteke sustava
├── home     maticni direktoriji korisnika
├── lib      biblioteke
├── tmp      privremene datoteke
├── usr      programi i podaci instalirani na sustavu
└── var      promjenjivi podaci (logovi, redovi poslova)
```

Jedno stablo, bez slova diskova

- ▶ UNIX **ne dijeli datotečni sustav prema diskovima**: nema C:, D:, E:.
- ▶ Različiti diskovi, particije i vanjski mediji **montiraju se** (*mount*) na proizvoljno mjesto u stablu — ne nužno na prvoj razini.
- ▶ Za korisnika je to potpuno transparentno: prelazak s jednog uređaja na drugi izgleda kao ulazak u poddirektorij.
- ▶ Isto vrijedi i za mrežne diskove — putanja izgleda jednako, neovisno o tome gdje podaci fizički leže.

Matični direktorij

- ▶ Svaki korisnik sustava ima vlastiti **matični direktorij** (*home*), obično `/home/ime_korisnika` (na macOS-u `/Users/ime_korisnika`).
- ▶ U njemu korisnik ima puna prava: tu drži svoje datoteke, postavke programa i radne direktorije.
- ▶ Nakon prijave na sustav to je **polazni radni direktorij**.
- ▶ Ljuska ga pamti u varijabli `$HOME`; naredba `cd` bez argumenta uvijek vraća u njega.
- ▶ Korisnik `root` iznimka je i po tome — njegov je matični direktorij `/root`, izvan `/home`.

Posebne oznake

Svaki direktorij pri stvaranju dobiva dva unosa koji se ne mogu izbrisati:

- ▶ `.` — **trenutni** direktorij,
- ▶ `..` — **roditeljski** direktorij.

Iznimka je korijenski direktorij `/`, gdje i `.` i `..` pokazuju na sam `/`.

Znak `~` u ljesci označava **matični direktorij** korisnika.

Apsolutna i relativna putanja

Apsolutna — počinje s /, vrijedi uvijek:

`/home/dkrst/nastava/unix`

Kao potpuna kućna adresa — razglednica stiže odakle god ju poslali.

Relativna — polazi od trenutnog direktorija:

```
nastava/unix      # iz /home/dkrst
../nastava/unix    # iz /home/dkrst/vjezbe
```

Kao uputa u gradu: “*na prvom križanju lijevo*” — vrijedi samo s određenog polazišta.

Kretanje po stablu

- ▶ `pwd` — ispis trenutnog radnog direktorija.
- ▶ `cd` — promjena direktorija; bez argumenta vraća u matični.

```
$ cd ~  
$ pwd  
/home/dkrst  
$ cd nastava/unix  
$ pwd  
/home/dkrst/nastava/unix  
$ cd ..  
$ pwd  
/home/dkrst/nastava
```

Struktura direktorija

Direktorij je datoteka koja sadrži popis zapisa o drugim datotekama:

```
/home/dkrst/work> ls -al
drwx----- 3 dkrst users 512 2008-03-10 .
drwx----- 4 dkrst users 512 2008-03-10 ..
-rwx----- 7 dkrst users 4608 2008-03-11 dat.txt
```

- ▶ Prva dva zapisa su . i .. — postoje u svakom direktoriju.
- ▶ Prvi znak retka označava **tip** datoteke: d za direktorij, - za običnu datoteku.
- ▶ Ostatak retka: prava pristupa, vlasnik, grupa, veličina, vrijeme izmjene i ime.

Tipovi datoteka

Oznaka	Tip	Opis
-	obična datoteka	tekst, program, slika, arhiva
d	direktorij	popis zapisa o datotekama
l	simbolički link	putanja na drugu datoteku
c	<i>character special</i>	uređaj znak po znak (/dev/tty)
b	<i>block special</i>	uređaj blok po blok (/dev/sda)
s	socket	krajnja točka komunikacije
p	FIFO	imenovani cjevovod

Tip se vidi kao **prvi znak** u ispisu naredbe `ls -l`.

Sve je datoteka

- ▶ UNIX-ova temeljna apstrakcija: **uređaji, komunikacijski kanali i mnogi drugi resursi prikazuju se kao datoteke.**
- ▶ Posljedica: svi se koriste istim malim skupom sistemskih poziva — `open()`, `read()`, `write()`, `close()`.
- ▶ Program ne mora znati radi li o datoteci na disku, terminalu, mrežnom socketu ili cjevovodu.
- ▶ Zato je UNIX programski jednostavan usprkos raznolikosti sklopovlja.

Prava pristupa

- ▶ UNIX je od početka **višekorisnički** — na istom računalu istovremeno radi više ljudi.
- ▶ Potreban je mehanizam koji sprječava da korisnici, slučajno ili namjerno, mijenjaju tuđe datoteke.
- ▶ Rješenje: svaka datoteka ima **vlasnika**, **grupu** i skup prava.

Tri skupine, tri prava

Skupine:

- ▶ **user** — vlasnik datoteke (obično onaj tko ju je stvorio),
- ▶ **group** — grupa kojoj datoteka pripada,
- ▶ **others** — svi ostali korisnici.

Prava:

Oznaka	Naziv	Značenje za datoteke
r	čitanje	otvaranje i čitanje sadržaja
w	pisanje	izmjena sadržaja
x	izvršavanje	pokretanje kao programa

Ukupno **devet bitova**: tri prava puta tri skupine.

Zapis prava

—	r	w	x	r	w	x	r	w	x
	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1	0/1
	4	2	1	4	2	1	4	2	1
	<i>user</i>			<i>group</i>			<i>others</i>		

Slika 1.3: Struktura prava pristupa (rwx) za vlasnika, grupu i ostale

- ▶ `rw-r--r--` — vlasnik čita i piše, grupa i ostali samo čitaju.
- ▶ `rw-r-x---` — vlasnik sve, grupa čita i izvršava, ostali ništa.

- ▶ Na Windowsima izvršnost određuje **ekstenzija** (.exe, .bat, .com).
- ▶ Na UNIX-u toga nema — jedini kriterij je **bit x**.
- ▶ Posljedice:
 - ▶ program.txt s postavljenim x je potpuno valjani izvršni program,
 - ▶ program.exe bez x ne pokreće se.
- ▶ Postavljanje x naravno ne čini bilo koju datoteku programom: ako sadržaj nije ni binarni kôd ni skripta, dobit ćemo grešku.

Nema ekstenzija

- ▶ Ime `program.c` ili `dat1.txt` — točka i nastavak su **samo dio imena**.
- ▶ Sustav **ne gleda nastavak** kad odlučuje o tipu datoteke ni o tome može li se izvršiti.
- ▶ Datoteka `dat1.txt` može biti tekst, binarni program, skripta ili direktorij.
- ▶ Praktična posljedica: uzorak `*.*` u UNIX-u znači “datoteke koje sadrže točku”, a ne “sve datoteke”. Za sve datoteke koristi se `*`.

Prava na direktorijima

Za direktorije ista slova imaju izmijenjeno značenje:

- ▶ r — pravo pregleda popisa datoteka (ls),
- ▶ w — pravo stvaranja, brisanja i preimenovanja datoteka **u** direktoriju,
- ▶ x — pravo “ulaska” u direktorij i pristupa datotekama u njemu.

Važno: brisanje datoteke kontrolira pravo w na **direktoriju**, a ne na samoj datoteci.

chmod — numerički način

Težine: $r = 4$, $w = 2$, $x = 1$. Tri znamenke: vlasnik, grupa, ostali.

```
chmod 644 dat1.txt      # rw-r--r--
chmod 750 skripta.sh     # rwxr-x---
chmod 600 tajna.txt      # rw-----
```

Numerički način **uvijek u potpunosti zamjenjuje** prethodna prava — treba navesti sva tri stupnja.

chmod — simbolički način

Skupine: u, g, o, a (sve). Operatori: + dodaj, - oduzmi.

```
chmod u+x skripta.sh      # vlasniku pravo izvršavanja
chmod go+rx skripta.sh    # grupi i ostalima citanje i izvršavanje
chmod a-x dat1.txt        # oduzmi izvršavanje svima
```

Mijenjaju se **samo navedena** prava; ostala ostaju nepromijenjena. Ako skupina već ima traženo pravo, naredba nema učinka.

- ▶ **root** (administrator, *superuser*) ima neograničene ovlasti: čita, mijenja i briše bilo koju datoteku, pokreće i zaustavlja bilo koji proces.
 - ▶ Koristiti s velikim oprezom — greška s root ovlastima može nepovratno oštetiti sustav.
 - ▶ Riječ “root” ima **dva različita značenja**:
 - ▶ administratorski račun (matični direktorij /root),
 - ▶ korijenski direktorij stabla (/).
 - ▶ Nemaju međusobne veze osim istog engleskog naziva.
-

Čak i ako na nekom računalu imate administratorske ovlasti, uobičajena je praksa da većinu vremena radite kao običan korisnik, a samo po potrebi naredbe izvršavate kao root.

Demo: pravo izvršavanja na datoteci

Cilj: pokazati da x, a ne ime, određuje izvršnost.

```
echo 'echo Pozdrav' > pozdrav.txt
./pozdrav.txt           # Permission denied
chmod u+x pozdrav.txt
./pozdrav.txt           # Pozdrav
ls -l pozdrav.txt
chmod 400 pozdrav.txt ; ls -l pozdrav.txt
```

Poanta: ista datoteka, isto ime — razlika je jedan bit.

Što smo naučili

- ▶ UNIX nastaje 1969. kao **jednostavniji odgovor na Multics**; oskudno sklopovlje, izostanak proračuna i besplatno dijeljenje s izvornim kodom oblikovali su sve ostalo.
- ▶ Danas ga nalazimo na poslužiteljima, telefonima, ugradbenim uređajima i **svim** superračunalima s liste TOP500.
- ▶ Arhitektura je slojevita; **sistemske pozivi su jedino sučelje** prema jezgri.
- ▶ **Sve je datoteka** — isti open/read/write/close za sve resurse.
- ▶ Jedno stablo s korijenom /, bez slova diskova i bez ekstenzija.
- ▶ Prava: tri skupine $\times$ tri prava; x određuje izvršnost, w na direktoriju određuje brisanje.

Za samostalan rad

Skripta, poglavlje 1 — zadaci za samostalno rješavanje:

1. **Prava pristupa i preusmjerenje**
2. **Ulančavanje naredbi**
3. **ocisti.sh**

Prvi zadatak možete riješiti već sada; za druga dva trebat će vam gradivo sljedećeg predavanja.

Naredbena ljuska i shell skripte

- ▶ preusmjeravanje ulaza i izlaza,
- ▶ ulančavanje naredbi,
- ▶ procesi u pozadini,
- ▶ pisanje skripti: varijable, petlje, argumenti, izlazni status.

Do tada: osigurajte si pristup UNIX okruženju i “prošecite” po datotečnom sustavu.